CS 499 Milestone Three: Enhancement Two – Algorithms and Data Structures Narrative

CS 499 Computer Science Capstone

Jared Borck

9/27/26

Artifact Description

The artifact I selected for my Algorithms and Data Structures enhancement is Travlr Getaways, a full-stack web application that I originally created in CS 465: Full Stack Development during the summer of 2026. The application uses Angular for the frontend, Node.js and Express for the backend and REST API, and MongoDB for data storage. During my previous CS 499 enhancement, I expanded the application with a customer rating and review system that allows authenticated users to rate trips in five categories: value, destination, overall quality, accommodations, and activities.

For this enhancement, I used the rating information created during the previous milestone to build a trip-ranking system. Instead of only storing and displaying individual ratings, the application can now process the review data and use it to calculate a weighted score for each trip. Overall quality contributes 30 percent of the score, destination contributes 25 percent, value contributes 20 percent, accommodations contributes 15 percent, and activities contributes 10 percent. The scores are then used to create a complete ranking of reviewed trips and identify the highest-rated trips.

The enhancement also adds several algorithm and data-structure components. Reviews are grouped by trip code using a JavaScript Map, the complete trip ranking is generated using a custom merge sort implementation, and the highest-rated trips are selected using a custom min-heap for a Top-K operation. The application includes a new rankings API endpoint and an Angular rankings page that displays the Top 3 trips, the complete ranking, the weighted scoring formula, and information about the algorithms used.

Justification for Inclusion

I chose to continue using Travlr Getaways because it allowed me to build on the work I had already completed while demonstrating a different area of computer science. The previous enhancement focused mainly on software engineering, security, authentication, authorization, and customer interaction. This enhancement uses the data created by that feature to solve a new problem: determining how trips should be ranked based on multiple customer rating categories.

The weighted scoring algorithm demonstrates how several pieces of information can be combined into one value that can be compared across trips. I decided not to treat every rating category equally because some categories can be more useful when determining the overall quality of a trip. Overall quality has the highest weight at 30 percent, followed by destination at 25 percent, value at 20 percent, accommodations at 15 percent, and activities at 10 percent. The algorithm calculates a weighted score for each individual review and then averages those results when a trip has multiple reviews.

I also used a Map to group reviews by trip code before calculating the rankings. Without this data structure, the application could repeatedly search through the entire review collection to find the reviews belonging to each trip. Building a lookup structure first makes the relationship between trips and their reviews easier to manage and avoids unnecessary repeated searches. This was a useful example of how choosing an appropriate data structure can improve the organization and efficiency of a solution.

For the complete ranking, I implemented merge sort instead of relying only on JavaScript's built-in sorting function. Merge sort provides predictable O(n log n) time complexity and demonstrates the process of dividing a problem into smaller portions, sorting those portions, and merging them back

together in the correct order. The comparison logic first compares weighted scores, then uses review count to resolve equal scores, and finally uses the trip name as another tie breaker.

The Top 3 feature uses a different approach. I implemented a min-heap to solve the Top-K problem. When the application only needs the best three trips, it is not necessary to fully sort every trip in the collection. The min-heap keeps only the strongest K candidates as the algorithm processes the ranking records. This approach has O(n log k) processing complexity and O(k) heap space. Comparing this approach with the O(n log n) complete sort gave me a practical example of how the best algorithm can depend on what information the program actually needs.

The artifact was also improved from the user's perspective. Travlr Getaways previously allowed users to submit and view reviews, but the ratings did not provide an easy way to compare trips. The new rankings page turns those individual ratings into information that can help a user identify highly rated trips and compare their overall results.

Course Outcome Coverage

This enhancement most directly demonstrates Course Outcome 3: designing and evaluating computing solutions using algorithmic principles and computer science practices while considering the trade-offs involved in design choices. This was the outcome I planned to address most directly with my Algorithms and Data Structures enhancement, and I believe the completed work provides strong evidence toward meeting it.

The enhancement required me to do more than simply sort a list. I considered different types of problems within the same feature and selected different approaches for them. Merge sort is appropriate when the application needs the complete ordered collection, while the min-heap is more appropriate when only the highest K results are needed. This allowed me to compare O(n log n) and O(n log k) approaches and consider the additional space each algorithm requires.

The enhancement also continues to support Course Outcome 4 because I used established tools and techniques to integrate the algorithms into an existing full-stack application. The algorithms are separated into their own utility module, while the controller retrieves the necessary data and uses those functions to create the API response. The Angular application then displays the processed results without duplicating the ranking logic on the frontend.

My overall outcome-coverage plan has not changed significantly. Course Outcomes 1 and 2 continue to be demonstrated more directly through my code review, written narratives, and final ePortfolio. Course Outcome 5 was demonstrated more strongly during my Software Design and Engineering enhancement through authentication, authorization, ownership checks, and validation. This milestone was intentionally focused on making Course Outcome 3 much more visible.

Reflection on the Enhancement Process

One of the main things I learned from this enhancement was that ranking data involves more design decisions than simply putting numbers in descending order. I first needed to decide how the different rating categories would contribute to the final score. After that, I had to determine how the review data

should be organized, how a complete ranking should be generated, and whether the same algorithm should be used when only a small number of top results were needed.

Implementing both merge sort and a min-heap helped make the trade-offs easier to understand. Merge sort makes sense when every reviewed trip needs to appear in ranked order because the complete collection has to be sorted. The Top-K problem is different. When only the three best trips are needed, maintaining a heap of three candidates avoids fully sorting the entire collection. Working with both approaches in the same application made the difference between theoretical complexity and a practical design decision much clearer to me.

Testing was also an important part of this enhancement. I initially had only one trip in the database with review information. The ranking page worked, but one result was not enough to demonstrate whether the sorting and Top-K algorithms were behaving correctly. I first tested the algorithms directly with a sample collection of five trips that had known scores. The merge sort correctly returned the trips in descending order, and the min-heap correctly returned the same three trips that appeared at the top of the complete ranking.

I then added additional customer accounts and review data so the actual application had multiple reviewed trips. This allowed me to test the complete process using real data from MongoDB. The final rankings page displayed four reviewed trips, while the Top 3 selected by the min-heap matched the first three results produced by the full merge sort. This helped confirm that the two algorithms were producing consistent results for the same dataset.

I also created automated tests for the algorithm module using Node's built-in test tools. The tests check weighted score calculations, averaging multiple reviews, empty review handling, grouping reviews with a Map, merge-sort ordering, tie-breaking behavior, Top-K selection, cases where K is larger than the available number of trips, and empty input. All 10 automated tests passed. Adding these tests gave me a repeatable way to verify the algorithms instead of relying only on visual inspection of the rankings page.

One smaller challenge occurred while adding test review data through PowerShell. Two trip names contained curly apostrophes while the script initially searched for straight apostrophes. Because of that difference, the trip codes were not found and those API requests returned 404 errors. I changed the search so it did not depend on the exact apostrophe character and then successfully added the remaining review data. Although this was not a problem with the ranking algorithms themselves, it reinforced the importance of checking the actual data being processed instead of assuming that visually similar strings are identical.

Overall, this enhancement changed Travlr Getaways from an application that simply collects customer ratings into one that can analyze those ratings and produce useful ranked results. More importantly, it gave me practical experience selecting and implementing data structures and algorithms based on the problem being solved. The use of a Map, weighted scoring, merge sort, and a min-heap allowed me to demonstrate several algorithmic concepts within a feature that has a clear purpose in the application. It also helped me better understand how complexity, data organization, testing, and design trade-offs influence the way a software solution is developed.